

Practical: 6

Implementation of chain matrix multiplication using dynamic programming

Aim: Implementation of chain matrix multiplication using dynamic programming

ALGORITHM:

Input: Dimension array $p[0 \dots n]$ for n matrices
output: Minimum number of Scalar multiplications

1. Start
2. Read the number of matrices n .
3. Read the dimension array $p[0 \dots n]$
4. Set $c[i][j] = 0$ for all matrices because multiplying one matrix needs no multiplication
5. Set chain length $d = 1$ to $n-1$.
6. for each d , calculate:
 - $i = 1$ to $n-d$
 - $j = i+d$
7. Set $min = 32767$.
8. Try Every possible split k from i to $j-1$
9. calculate the cost:
$$q = c[i][k] + c[k+1][j] + p[i-1] \times p[k] \times p[j]$$
10. if $q < min$, update:
 - $min = q$
 - $s[i][j] = k$
11. Stop the minimum cost:
$$c[i][j] = min$$
12. The final minimum cost is $c[1][n]$.
13. Stop.

PROGRAM

```
#include <iostream>
#include <climits>
using namespace std;

void printTable(int m[][100], int n)
{
    cout << "\nDP Table (Minimum Costs):\n";

    for (int i = 1; i < n; i++)
    {
        for (int j = 1; j < n; j++)
        {
            if (i > j)
                cout << "-\t";
            else
                cout << m[i][j] << "\t";
        }
        cout << endl;
    }
}

int matrixChainOrder(int p[], int n)
{
    int m[100][100];

    for (int i = 1; i < n; i++)
    {
        m[i][i] = 0;
    }

    for (int L = 2; L < n; L++)
    {
        for (int i = 1; i < n - L + 1; i++)
        {
            int j = i + L - 1;

            // Set initial cost to infinity
            m[i][j] = INT_MAX;
```

```
        for (int k = i; k < j; k++)
        {
            int q = m[i][k]
                + m[k + 1][j]
                + p[i - 1] * p[k] * p[j];

            if (q < m[i][j])
            {
                m[i][j] = q;
            }
        }
    }
}

printTable(m, n);

return m[1][n - 1];
}

// Main function
int main()
{
    // Dimensions of matrices:
    // A1 = 3 x 5
    // A2 = 5 x 10
    // A3 = 10 x 7
    // A4 = 7 x 2
    int arr[] = {3, 5, 10, 7, 2};

    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Minimum number of
multiplications is: "
        << matrixChainOrder(arr, n);

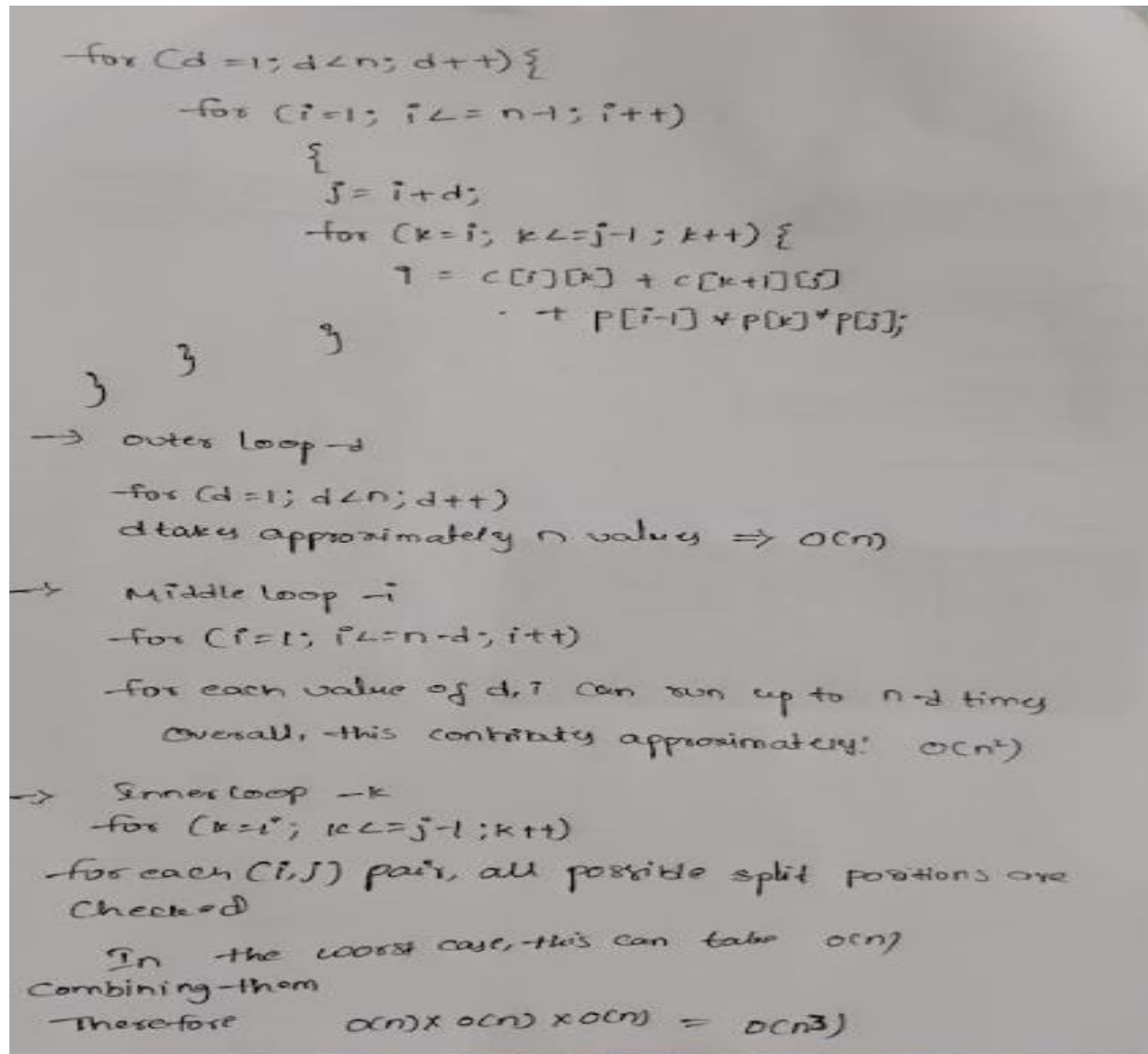
    return 0;
}
```

OUTPUT:

```
Minimum number of multiplications is
DP Table (Minimum Costs):
0      150      360      270
- 0      350      240
- - 0 140
- - - 0
270

...Program finished with exit code 0
Press ENTER to exit console.
```

TIME COMPLEXITY:



```

for (d = 1; d < n; d++) {
    for (i = 1; i <= n - d; i++) {
        {
            j = i + d;
            for (k = i; k <= j - 1; k++) {
                T = C[i][k] + C[k+1][j]
                + P[i-1] * P[k] * P[j];
            }
        }
    }
}

```

→ outer loop - d
 for (d = 1; d < n; d++)
 d takes approximately n values $\Rightarrow O(n)$

→ Middle loop - i
 for (i = 1; i <= n - d; i++)
 for each value of d, i can run up to n - d times
 overall, this contributes approximately: $O(n^2)$

→ Inner loop - k
 for (k = i; k <= j - 1; k++)
 for each (i, j) pair, all possible split positions are checked
 In the worst case, this can take $O(n)$
 Combining them
 Therefore $O(n) \times O(n) \times O(n) = O(n^3)$

CONCLUSION:

Matrix Chain Multiplication using Dynamic Programming finds the optimal order of multiplying matrices with the minimum number of scalar multiplications. It avoids repeated calculations by storing previous results in a DP table.